LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 5 – Relasi antar Kelas



NAMA: NISA

NIM: 103132400044

KELAS: PS1DS-05-REG-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Pemrograman Berorientasi Objek (Object-Oriented Programming atau OOP) merupakan paradigma pemrograman yang menggunakan kelas dan objek untuk membangun suatu sistem. Dalam OOP, kelas berfungsi sebagai cetak biru (blueprint) yang mendefinisikan atribut dan metode yang dimiliki oleh objek. Salah satu konsep penting dalam OOP adalah relasi antar kelas, yaitu hubungan yang terbentuk antara dua kelas atau lebih untuk mendukung interaksi dan pengorganisasian program. Relasi antar kelas dapat digambarkan menggunakan diagram kelas (class diagram) yang menunjukkan nama kelas, atribut, metode, serta hubungan antar kelas.

Relasi antar kelas terdiri atas beberapa jenis, di antaranya asosiasi, agregasi, dan komposisi. Asosiasi merupakan hubungan antara dua kelas yang memungkinkan objek dari masing-masing kelas saling berinteraksi, contohnya hubungan antara kelas Dosen dan Mahasiswa. Agregasi merupakan hubungan antara bagian (part) dan keseluruhan (whole), tetapi objek dari kelas bagian tetap dapat berdiri sendiri tanpa bergantung pada keberadaan kelas keseluruhan, contohnya hubungan antara kelas Jurusan dan Mahasiswa. Sementara itu, komposisi merupakan bentuk hubungan yang lebih kuat dibandingkan agregasi, karena keberadaan objek bagian bergantung pada objek keseluruhan, seperti hubungan antara kelas Komputer dengan CPU, Monitor, dan Mouse.

Dalam bahasa pemrograman Java, relasi antar kelas dapat diimplementasikan dengan menggunakan objek, atribut, metode, dan konstruktor sesuai kebutuhan hubungan yang dibentuk. Asosiasi dapat diterapkan melalui pemanggilan metode objek dari kelas lain, agregasi dapat diterapkan dengan menyimpan referensi objek lain sebagai bagian dari suatu kelas, sedangkan komposisi dapat diterapkan dengan membentuk objek bagian sebagai komponen yang bergantung pada objek utama. Pemahaman mengenai relasi antar kelas sangat penting untuk menghasilkan program yang terstruktur, mempermudah pengelolaan kode, serta mendukung penerapan konsep OOP dalam pengembangan perangkat lunak.

B. Guided

a. Guided Asosiasi

Mahasiswa.java

```
package guided.asosiasi;

public class Mahasiswa {
    private String nim;
    private String nama;
    private int nilai;

    public void setNim(String nim) {
        this.nim = nim;
    }

    public void setNama(String nama) {
        this.nama = nama;
    }

    public void setNilai(int nilai) {
        this.nilai = nilai;
    }

    public String getNim() {
        return this.nim;
    }

    public String getNama() {
        return this.nama;
    }
```

```
    public int getNilai() {
        return this.nilai;
    }
}
```

Dosen.java

```
package guided.asosiasi;

public class Dosen {
    private String kodeDosen;
    private String namaDosen;

    // Setter
    public void setKodeDosen(String kodeDosen) {
        this.kodeDosen = kodeDosen;
    }

    public void setNamaDosen(String namaDosen) {
        this.namaDosen = namaDosen;
    }

    // Getter
    public String getKodeDosen() {
        return this.kodeDosen;
    }
```

```
    public String getNamaDosen() {
        return this.namaDosen;
    }

    public void giveScore(Mahasiswa s, int nilai) {
        // Asosiasi: memanggil method dari objek Mahasiswa
        s.setNilai(nilai);
    }

    public int getScore(Mahasiswa s) {
        // Mengambil nilai dari objek Mahasiswa
        return s.getNilai();
    }

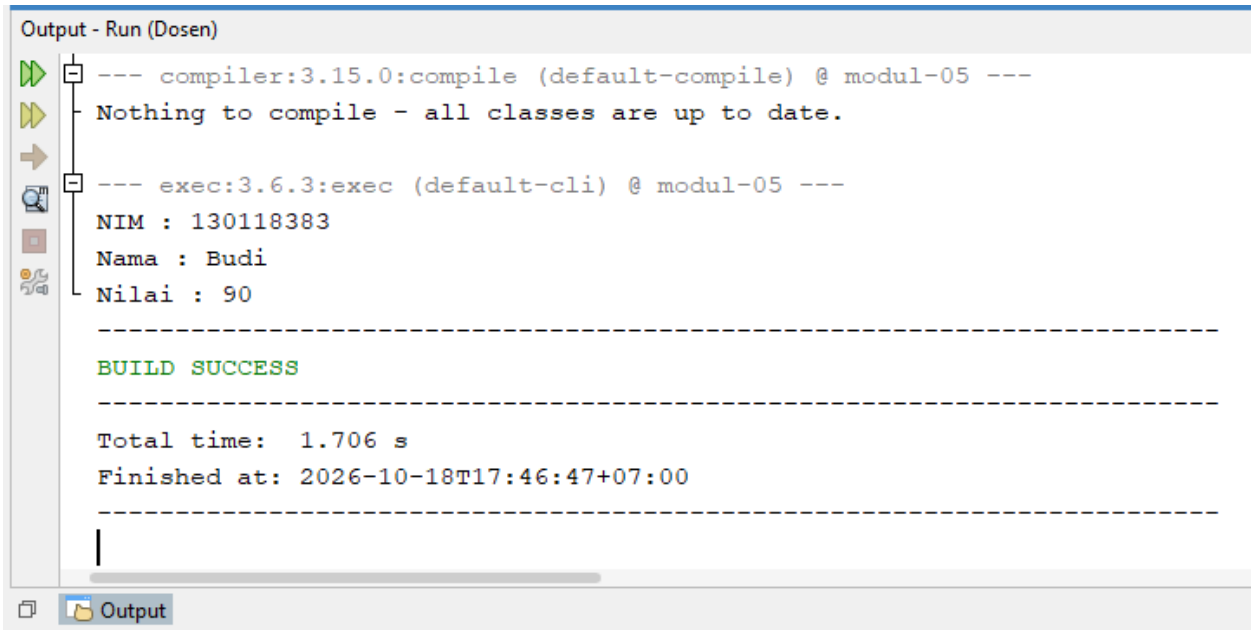
    public static void main(String[] args) {
        Mahasiswa m = new Mahasiswa();

        m.setNim("130118383");
        m.setNama("Budi");

        Dosen d = new Dosen();
        d.giveScore(m, 90);

        System.out.println("NIM : " + m.getNim());
        System.out.println("Nama : " + m.getNama());
        System.out.println("Nilai : " + d.getScore(m));
    }
}
```

Output :



b. Guided agregasi

Mahasiswa.java

```
package guided.agregasi;

public class Mahasiswa {
    private String NIM, Nama;

    public Mahasiswa(String NIM, String Nama){
        this.NIM = NIM;
        this.Nama = Nama;
    }

    //Getter
    public String getNIM(){
        return NIM;
    }
    public String getNama(){
        return Nama;
    }

    //Setter
    public void setNIM(String NIM){
        this.NIM = NIM;
    }
    public void setNama(String Nama){
        this.Nama = Nama;
    }
```

```
}
```

Jurusan.java

```
package guided.agregasi;

public class Jurusan {
    private static int jumlahMahasiswa;
    private String kodeJurusan, namaJurusan;
    private Mahasiswa[] daftarMahasiswa = new Mahasiswa[32];

    //constructor
    public Jurusan (String kodeJurusan, String namaJurusan){
        this.kodeJurusan = kodeJurusan;
        this.namaJurusan = namaJurusan;
    }

    //Behaviour
    public void addMahasiswa(Mahasiswa m){
        this.daftarMahasiswa[jumlahMahasiswa] = m;
        jumlahMahasiswa++;
    }

    public void displayMahasiswa(){
        System.out.println("Kode Jurusan: " + this.kodeJurusan);
        System.out.println("Nama Jurusan: " + this.namaJurusan);

        System.out.println("Daftar Mahasiswa: ");

         for (int i = 0; i < jumlahMahasiswa; i++) {
            System.out.println((i + 1) + ". "
                    + daftarMahasiswa[i].getNIM() + " "
                    + daftarMahasiswa[i].getNama());
        }
    }

     // Getter
    public String getKodeJurusan() {
        return this.kodeJurusan;
    }

    public String getNamaJurusan() {
        return this.namaJurusan;
    }

    // Setter
    public void setKodeJurusan(String kodeJurusan) {
        this.kodeJurusan = kodeJurusan;
    }
```

```
    public void setNamaJurusan(String namaJurusan) {
        this.namaJurusan = namaJurusan;
    }

}
```

c. Guided Komposisi

Processor.java

```
package guided.komposisi;

public class Processor {

    private String brand;
    private int kecepatan;

    //constructor
    public Processor(String brand, int kecepatan) {
        this.brand = brand;
        this.kecepatan = kecepatan;
    }

    //behaviour
    public void displaySpec() {
        System.out.println(this.brand + ", " + this.kecepatan);
    }
}
```

Monitor.java

```
package guided.komposisi;

public class Monitor {

    private String brand;

    public Monitor(String brand) {
        this.brand = brand;
    }

    public void DisplaySpec() {
        System.out.println(this.brand);
    }
}
```

Mouse.java

```
package guided.komposisi;

public class Mouse {

    private String brand, tipe;

    public Mouse(String brand, String tipe) {
        this.brand = brand;
        this.tipe = tipe;
    }

    public void displaySpec() {
        System.out.println(this.brand + ", " + this.tipe);
    }
}
```

Komputer.java

```
package guided.komposisi;

public class Komputer {
     private Processor processor;
    private Monitor monitor;
    private Mouse mouse;

    public Komputer(String brandCpu, int kecepatanCpu, String brandMonitor, String
brandMouse, String tipeMouse){
        this.processor = new Processor(brandCpu, kecepatanCpu);
        this.monitor = new Monitor(brandMonitor);
        this.mouse = new Mouse(brandMouse, tipeMouse);
    }

    public void displaySpecKomputer() {
        System.out.println("=== SPESIFIKASI KOMPUTER ===");
        this.processor.displaySpec();
        this.monitor.DisplaySpec();
        this.mouse.displaySpec();
    }

}
```

Main.java

```
import guided.agregasi.*;
import guided.komposisi.*;

public class Main {
```

```
    public static void main(String[] args){
        runGuidedAgregrasi();
        runGuidedKomposisi();
    }

    public static void runGuidedAgregrasi(){
        Mahasiswa m1 = new Mahasiswa("103112400148", "Rendhi");
        Mahasiswa m2 = new Mahasiswa("103112400149", "John");

        Jurusan prodi = new Jurusan("SD","Sains Data");

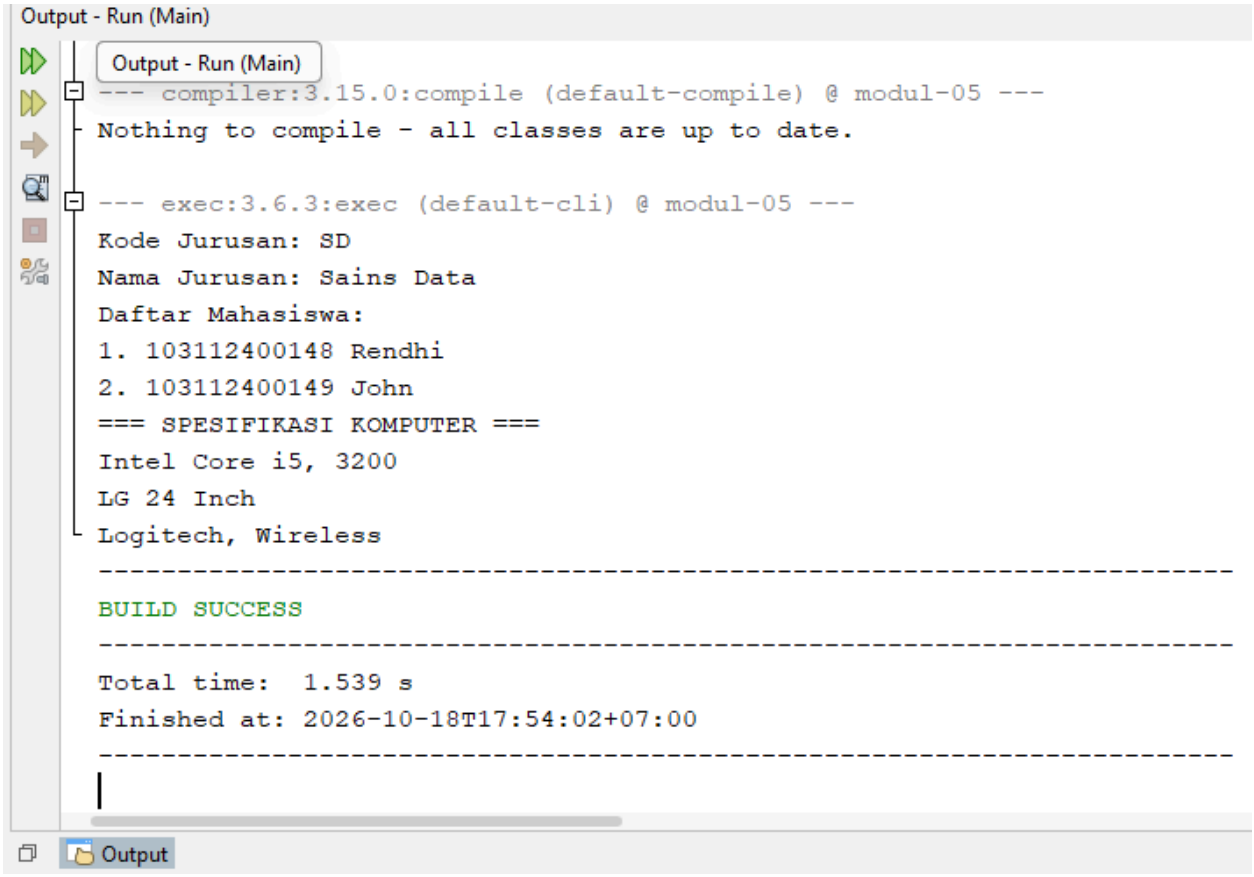
        prodi.addMahasiswa(m1);
        prodi.addMahasiswa(m2);

        prodi.displayMahasiswa();


    }

    public static void runGuidedKomposisi(){
        Komputer pc = new Komputer("Intel Core i5",3200,"LG 24
Inch","Logitech","Wireless");
        pc.displaySpecKomputer();
    }
}
```

Output :



C. Unguided

1. Soal Unguided

Buatlah program Java bertema Sistem Manajemen Pesanan Cafe yang menerapkan tiga jenis relasi antar kelas, yaitu komposisi (composition), agregasi (aggregation), dan asosiasi (association). Program terdiri dari class `RincianHarga` untuk menghitung harga dasar, pajak sebesar 10%, dan harga total; class `ItemPesanan` untuk menyimpan nama item dan rincian harga; class `Pesanan` untuk menampung beberapa objek item menggunakan array standar; class `Kasir` untuk memproses pembayaran dan mencetak struk; serta class `Main` untuk menjalankan program. Buatlah objek kasir bernama Budi dan tiga objek item pesanan, yaitu Iced Americano dengan harga dasar Rp18.000, Beef Croissant dengan harga dasar Rp25.000, dan Matcha Latte dengan harga dasar Rp22.000. Selanjutnya, buat pesanan untuk meja 4 dengan kapasitas array sebanyak lima item, masukkan ketiga item ke dalam pesanan, tampilkan daftar pesanan beserta harga dasar, pajak, dan harga total setiap item, lalu proses pembayaran dan

tampilkan struk yang berisi subtotal, total pajak, dan total pembayaran. Dalam pengerjaannya, terapkan enkapsulasi dengan menggunakan `private`, constructor, dan getter. Gunakan nama parameter constructor yang sama dengan nama field sehingga perlu menggunakan kata kunci `this`. Objek `RincianHarga` wajib dibuat di dalam constructor `ItemPesanan`, sedangkan objek `ItemPesanan` harus dibuat secara independen di `Main` sebelum dimasukkan ke dalam `Pesanan`. Class `Kasir` tidak boleh menyimpan objek `Pesanan` sebagai field dan hanya boleh berinteraksi dengannya melalui parameter method `prosesPembayaran()`. Gunakan array standar bertipe tetap, bukan `ArrayList`, serta jangan menggunakan inheritance, overriding, maupun abstract class. Hasil program harus menunjukkan subtotal Rp65.000, total pajak Rp6.500, dan total pembayaran Rp71.500.

Jawaban :

ItemPesanan.java

```
package unguided;


public class ItemPesanan {
    private String namaItem;
    private RincianHarga rincian;


    public ItemPesanan(String namaItem, double hargaDasar) {
        this.namaItem = namaItem;
        this.rincian = new RincianHarga(hargaDasar);
    }


    public String getNamaItem() {
        return namaItem;
    }


    public RincianHarga getRincian() {
```

```
        return rincian;
    }
}
```

RincianHarga.java

```
package unguided;


public class RincianHarga {
    private double hargaDasar;
    private double pajak;


    public RincianHarga(double hargaDasar) {
        this.hargaDasar = hargaDasar;
        this.pajak = hargaDasar * 0.10;
    }


    public double getHargaDasar() {
        return hargaDasar;
    }


    public double getPajak() {
        return pajak;
    }


    public double getHargaTotal() {
        return hargaDasar + pajak;
    }
}
```

Pesanan.java

```
package unguided;


public class Pesanan {
    private int nomorMeja;
    private ItemPesanan[] daftarItem;
    private int jumlahItem;

    public Pesanan(int nomorMeja, int kapasitas) {
        this.nomorMeja = nomorMeja;
        this.daftarItem = new ItemPesanan[kapasitas];
        this.jumlahItem = 0;
    }

    public void tambahItem(ItemPesanan item) {
        if (jumlahItem < daftarItem.length) {
            daftarItem[jumlahItem] = item;
            jumlahItem++;
        } else {
            System.out.println("Pesanan sudah penuh!");
        }
    }

    public void tampilkanPesanan() {
        System.out.println("=== DAFTAR PESANAN CAFE ===");
        System.out.println("Nomor Meja: " + nomorMeja);
```

```
        for (int i = 0; i < jumlahItem; i++) {
            ItemPesanan item = daftarItem[i];
            RincianHarga rincian = item.getRincian();

            System.out.println(
                (i + 1) + ". " + item.getNamaItem()
                + " - Rp " + rincian.getHargaTotal()
                + " (Harga: " + rincian.getHargaDasar()
                + ", Pajak: " + rincian.getPajak() + ")"
            );
        }

        System.out.println();
    }

    public int getNomorMeja() {
        return nomorMeja;
    }

    public ItemPesanan[] getDaftarItem() {
        return daftarItem;
    }

    public int getJumlahItem() {
        return jumlahItem;
    }
}
```

Kasir.java

```
package unguided;

public class Kasir {
    private String namaKasir;

    public Kasir(String namaKasir) {
        this.namaKasir = namaKasir;
    }

    public void prosesPembayaran(Pesanan pesanan) {
        double totalSubtotal = 0;
        double totalPajak = 0;

        ItemPesanan[] daftarItem = pesanan.getDaftarItem();

        for (int i = 0; i < pesanan.getJumlahItem(); i++) {
            RincianHarga rincian = daftarItem[i].getRincian();

            totalSubtotal += rincian.getHargaDasar();
            totalPajak += rincian.getPajak();
        }

        double totalBayar = totalSubtotal + totalPajak;

        System.out.println("=== STRUK PEMBAYARAN KASIR ===");
        System.out.println("Kasir Bertugas : " + namaKasir);
        System.out.println("Pesanan        : Meja "
```

```
            + pesanan.getNomorMeja());
        System.out.println("Jumlah Item    : "
            + pesanan.getJumlahItem());
        System.out.println("Total Subtotal : Rp " + totalSubtotal);
        System.out.println("Total Pajak    : Rp " + totalPajak);
        System.out.println("TOTAL BAYAR    : Rp " + totalBayar);
        System.out.println("==============================");
    }
}
```

Main.java

```
package unguided;


public class Main {


    public static void main(String[] args) {
        // Membuat objek kasir
        Kasir kasir = new Kasir("Budi");


        // Membuat objek item pesanan secara independen
        ItemPesanan item1 = new ItemPesanan(
            "Iced Americano", 18000
        );


        ItemPesanan item2 = new ItemPesanan(
            "Beef Croissant", 25000
        );
```

```
        ItemPesanan item3 = new ItemPesanan(
                "Matcha Latte", 22000
        );

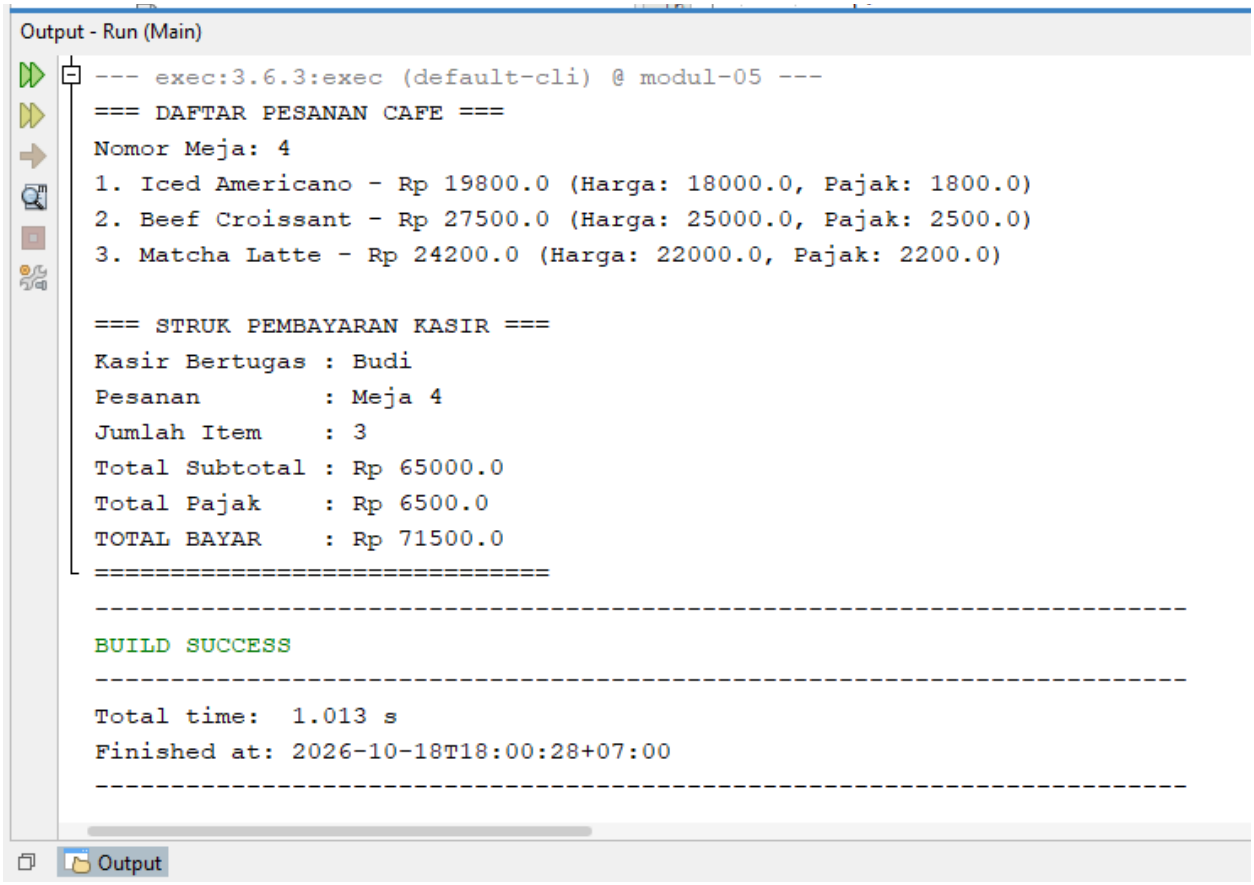
        // Membuat pesanan untuk meja 4 dengan kapasitas 5 item
        Pesanan pesanan = new Pesanan(4, 5);

        // Menambahkan item ke dalam pesanan
        pesanan.tambahItem(item1);
        pesanan.tambahItem(item2);
        pesanan.tambahItem(item3);

        // Menampilkan daftar pesanan
        pesanan.tampilkanPesanan();

        // Memproses pembayaran
        kasir.prosesPembayaran(pesanan);
    }
}
```

Output :



Penjelasan :

Proyek Sistem Manajemen Pesanan Cafe ini dibuat menggunakan Java untuk menerapkan konsep relasi antar kelas, yaitu komposisi, agregasi, dan asosiasi. Program terdiri dari lima class yang saling berinteraksi untuk mengelola item pesanan, menghitung pajak sebesar 10%, serta memproses pembayaran dan menampilkan struk. Dalam pembuatannya, program menerapkan enkapsulasi, constructor, getter, dan array standar untuk menyimpan daftar item. Hasil akhirnya berupa daftar pesanan dan struk pembayaran dengan total bayar sebesar Rp71.500.

D. Kesimpulan

Berdasarkan pembelajaran Bab 5 tentang relasi antar kelas, dapat disimpulkan bahwa dalam pemrograman berorientasi objek (OOP), setiap kelas dapat saling berhubungan untuk membentuk suatu sistem yang terstruktur. Terdapat tiga jenis relasi yang dipelajari, yaitu asosiasi sebagai hubungan interaksi antarobjek, agregasi sebagai hubungan bagian dan keseluruhan yang objeknya tetap dapat berdiri sendiri, serta komposisi sebagai hubungan yang lebih kuat karena keberadaan objek bagian bergantung

pada objek utamanya. Melalui praktikum ini, saya memahami cara menerapkan relasi antar kelas dalam Java menggunakan atribut, constructor, method, dan objek. Penerapan konsep tersebut membantu membuat program menjadi lebih terorganisasi dan memudahkan pengelolaan data serta interaksi antar kelas. Dengan demikian, pemahaman tentang relasi antar kelas menjadi dasar penting dalam merancang program berbasis objek yang terstruktur dan sesuai kebutuhan.